Data Structures Lab – Lab 1

Arrays, Linear Search & Binary Search

Sanamdeep Singh
Roll No: 2024BCSE027

Department of Computer Science & Engineering
NIT Srinagar

March 24, 2026

Contents

1 Part A: Array Problems **2**
1.1 Problem A1 – Find Maximum and Minimum in an Array 2
1.2 Problem A2 – Rotate an Array by K Positions 4
1.3 Problem A3 – Count Occurrences of Each Element 6

2 Part B: Linear Search Problems **8**
2.1 Problem B1 – Find All Occurrences Using Linear Search 8
2.2 Problem B2 – Search in a 2D Matrix Using Linear Search 10
2.3 Problem B3 – Find the Second Largest Element 12

3 Part C: Binary Search Problems **14**
3.1 Problem C1 – Find First and Last Position of Element 14
3.2 Problem C2 – Count Elements in a Range Using Binary Search 16
3.3 Problem C3 – Find Square Root Using Binary Search 18

1 Part A: Array Problems

1.1 Problem A1 – Find Maximum and Minimum in an Array

Difficulty: Easy

Problem Statement

Given an integer array `arr` of size n, find and print the maximum and minimum elements without using any built-in sorting or max/min functions. Solution must use a single pass (one loop only).

Pseudocode

```
SET max = arr[0], min = arr[0]
SET maxIndex = 0, minIndex = 0
FOR i = 1 TO n-1 DO
    IF arr[i] > max THEN
        max = arr[i], maxIndex = i
    IF arr[i] < min THEN
        min = arr[i], minIndex = i
PRINT max, min with their indices
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {34, 7, 23, 32, 5, 62, 15, 8};
    int n = 8;

    int max = arr[0];
    int min = arr[0];
    int maxIndex = 0;
    int minIndex = 0;

    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
            maxIndex = i;
        }
        if (arr[i] < min) {
            min = arr[i];
            minIndex = i;
        }
    }

    cout << "Maximum element: " << max
         << " (at index " << maxIndex << ")" << endl;
    cout << "Minimum element: " << min
         << " (at index " << minIndex << ")" << endl;

    return 0;
}
```

Output

```
Maximum element: 62 (at index 5)
Minimum element: 5 (at index 4)
```

Complexity Analysis

- **Time Complexity:** $O(n)$ – single pass through the array.
- **Space Complexity:** $O(1)$ – only constant extra variables used.

1.2 Problem A2 – Rotate an Array by K Positions

Difficulty: Medium

Problem Statement

Given an integer array of size n and integer k, rotate the array to the right by k positions. Implement both: (1) extra array approach $O(n)$ space, (2) in-place reversal trick $O(1)$ space.

Pseudocode

```
// Approach 1: Extra Array
SET k = k MOD n
FOR i = 0 TO n-1 DO
    temp[(i + k) MOD n] = arr[i]
COPY temp back to arr

// Approach 2: Reversal Trick
SET k = k MOD n
Step 1: Reverse entire array
Step 2: Reverse first k elements
Step 3: Reverse remaining n-k elements
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {1, 2, 3, 4, 5, 6, 7};
    int n = 7;
    int k = 3;

    k = k % n;  // handle k > n

    cout << "Original array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;

    // === Approach 1: Extra Array ===
    int temp[100];
    for (int i = 0; i < n; i++)
        temp[(i + k) % n] = arr[i];
    for (int i = 0; i < n; i++)
        arr[i] = temp[i];

    cout << "Rotated (extra array): ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;

    // === Approach 2: Reversal Trick ===
    int arr2[] = {1, 2, 3, 4, 5, 6, 7};
```

```

    // Step 1: Reverse entire array
    int start = 0, end = n - 1;
    while (start < end) {
        int t = arr2[start];
        arr2[start] = arr2[end];
        arr2[end] = t;
        start++;
        end--;
    }

    // Step 2: Reverse first k elements
    start = 0; end = k - 1;
    while (start < end) {
        int t = arr2[start];
        arr2[start] = arr2[end];
        arr2[end] = t;
        start++;
        end--;
    }

    // Step 3: Reverse remaining n-k elements
    start = k; end = n - 1;
    while (start < end) {
        int t = arr2[start];
        arr2[start] = arr2[end];
        arr2[end] = t;
        start++;
        end--;
    }

    cout << "Rotated (reversal trick): ";
    for (int i = 0; i < n; i++)
        cout << arr2[i] << " ";
    cout << endl;

    return 0;
}
```

Output

```
Original array: 1 2 3 4 5 6 7
Rotated (extra array): 5 6 7 1 2 3 4
Rotated (reversal trick): 5 6 7 1 2 3 4
```

Complexity Analysis

Approach	Time	Space
Extra Array	$O(n)$	$O(n)$
Reversal Trick	$O(n)$	$O(1)$

1.3 Problem A3 – Count Occurrences of Each Element

Difficulty: Medium

Problem Statement

Given an integer array of size n, count and display the frequency of each unique element, sorted by element value.

Pseudocode

```
SORT the array using bubble sort
SET i = 0
WHILE i < n DO
    SET count = 1
    WHILE arr[i + count] == arr[i] DO
        count = count + 1
    PRINT arr[i] and count
    i = i + count
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {4, 2, 6, 2, 4, 4, 1, 6, 2, 8};
    int n = 10;

    // Bubble sort
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    // Count and print frequencies
    cout << "Element\tFrequency" << endl;
    cout << "-------\t---------" << endl;

    int i = 0;
    while (i < n) {
        int count = 1;
        while (i + count < n && arr[i + count] == arr[i])
            count++;
        cout << arr[i] << "\t" << count << endl;
        i = i + count;
    }

    return 0;
}
```

Output

```
Element Frequency
------- ---------
1       1
2       3
4       3
6       2
8       1
```

Complexity Analysis

- **Time Complexity:** $O(n^2)$ due to bubble sort.
- **Space Complexity:** $O(1)$ – sorting done in-place.

2 Part B: Linear Search Problems

2.1 Problem B1 – Find All Occurrences Using Linear Search

Difficulty: Easy

Problem Statement

Modify linear search to find and return all indices where a target element appears. Also print percentage occurrence (bonus).

Pseudocode

```
SET count = 0
FOR i = 0 TO n-1 DO
    IF arr[i] == target THEN
        SAVE i in indices array
        count = count + 1
PRINT all indices, count
PRINT percentage = (count / n) * 100
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {4, 2, 7, 2, 9, 2, 5, 2};
    int n = 8;
    int target = 2;

    int indices[100];
    int count = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            indices[count] = i;
            count++;
        }
    }

    if (count == 0) {
        cout << "Target " << target
             << " not found." << endl;
    } else {
        cout << "Target " << target
             << " found at indices: [";
        for (int i = 0; i < count; i++) {
            cout << indices[i];
            if (i < count - 1)
                cout << ", ";
        }
        cout << "]" << endl;
        cout << "Total occurrences: " << count << endl;

```

```
        float percentage = (float)count / n * 100;
        cout << "Percentage: " << percentage
             << "%" << endl;
    }

    return 0;
}
```

Output

```
Target 2 found at indices: [1, 3, 5, 7]
Total occurrences: 4
Percentage: 50%
```

Complexity Analysis

- **Time Complexity:** $O(n)$ – checks every element.
- **Space Complexity:** $O(k)$ where k = number of occurrences.

2.2 Problem B2 – Search in a 2D Matrix Using Linear Search

Difficulty: Medium

Problem Statement

Given a 2D matrix and a target value, use linear search to find the target. Return row and column index if found, or $(-1, -1)$ if not.

Pseudocode

```
SET foundRow = -1, foundCol = -1
FOR i = 0 TO rows-1 DO
    FOR j = 0 TO cols-1 DO
        IF matrix[i][j] == target THEN
            foundRow = i, foundCol = j
PRINT foundRow, foundCol
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int matrix[3][3] = {
        {10, 20, 30},
        {40, 50, 60},
        {70, 80, 90}
    };
    int rows = 3;
    int cols = 3;
    int target = 50;

    int foundRow = -1;
    int foundCol = -1;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (matrix[i][j] == target) {
                foundRow = i;
                foundCol = j;
            }
        }
    }

    if (foundRow == -1) {
        cout << "Target " << target
             << " not found." << endl;
        cout << "Position: (-1, -1)" << endl;
    } else {
        cout << "Target " << target
             << " found at position: Row "
             << foundRow << ", Column "
             << foundCol << endl;
    }
```

```

    return 0;
}
```

Output

```
Target 50 found at position: Row 1, Column 1
```

Complexity Analysis

- **Time Complexity:** $O(m \times n)$ – checks every element.
- **Space Complexity:** $O(1)$ – constant extra space.

2.3 Problem B3 – Find the Second Largest Element

Difficulty: Medium

Problem Statement

Using linear search, find the second largest unique element. Handle duplicates and edge cases.

Pseudocode

```
// First loop: find the largest
SET largest = arr[0], largestIndex = 0
FOR i = 1 TO n-1 DO
    IF arr[i] > largest THEN
        largest = arr[i], largestIndex = i

// Second loop: find second largest (different from largest)
SET secondLargest = -1, found = false
FOR i = 0 TO n-1 DO
    IF arr[i] != largest THEN
        IF found == false OR arr[i] > secondLargest THEN
            secondLargest = arr[i], secondIndex = i
            found = true

PRINT largest and secondLargest
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {12, 35, 1, 10, 34, 1};
    int n = 6;

    // First find the largest
    int largest = arr[0];
    int largestIndex = 0;

    for (int i = 1; i < n; i++) {
        if (arr[i] > largest) {
            largest = arr[i];
            largestIndex = i;
        }
    }

    // Find second largest (different from largest)
    int secondLargest = -1;
    int secondIndex = -1;
    bool found = false;

    for (int i = 0; i < n; i++) {
        if (arr[i] != largest) {
            if (found == false) {
```

```
                    secondLargest = arr[i];
                    secondIndex = i;
                    found = true;
                } else if (arr[i] > secondLargest) {
                    secondLargest = arr[i];
                    secondIndex = i;
                }
            }
        }

        cout << "Largest element: " << largest
             << " (at index " << largestIndex
             << ")" << endl;

        if (found == false) {
            cout << "No second largest "
                 << "(all elements are same)."
                 << endl;
        } else {
            cout << "Second largest element: "
                 << secondLargest
                 << " (at index " << secondIndex
                 << ")" << endl;
        }

        return 0;
    }
```

Output

```
Largest element: 35 (at index 1)
Second largest element: 34 (at index 4)
```

Complexity Analysis

- **Time Complexity:** $O(n)$ – two passes through the array.
- **Space Complexity:** $O(1)$ – constant extra space.

3 Part C: Binary Search Problems

3.1 Problem C1 – Find First and Last Position of Element

Difficulty: Medium

Problem Statement

Given a sorted array with duplicates and a target, use binary search to find the first and last position. Return $[-1, -1]$ if not found.

Pseudocode

```
// Find first occurrence
SET low = 0, high = n-1, first = -1
WHILE low <= high DO
    mid = (low + high) / 2
    IF arr[mid] == target THEN
        first = mid
        high = mid - 1        // keep searching left
    ELSE IF arr[mid] < target THEN
        low = mid + 1
    ELSE
        high = mid - 1

// Find last occurrence (same logic but search right)
SET low = 0, high = n-1, last = -1
WHILE low <= high DO
    mid = (low + high) / 2
    IF arr[mid] == target THEN
        last = mid
        low = mid + 1         // keep searching right
    ELSE IF arr[mid] < target THEN
        low = mid + 1
    ELSE
        high = mid - 1
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {2, 4, 4, 4, 6, 8, 8, 10};
    int n = 8;
    int target = 4;

    // Find First Occurrence
    int low = 0, high = n - 1;
    int first = -1;

    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) {
            first = mid;
```

```
            high = mid - 1;  // search left
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    // Find Last Occurrence
    low = 0;
    high = n - 1;
    int last = -1;

    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) {
            last = mid;
            low = mid + 1;  // search right
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    if (first == -1) {
        cout << "Target " << target
             << " not found." << endl;
        cout << "Result: [-1, -1]" << endl;
    } else {
        cout << "First occurrence of "
             << target << ": index "
             << first << endl;
        cout << "Last occurrence of "
             << target << ": index "
             << last << endl;
    }

    return 0;
}
```

Output

```
First occurrence of 4: index 1
Last occurrence of 4: index 3
```

Complexity Analysis

- **Time Complexity:** $O(\log n)$ – two binary searches.
- **Space Complexity:** $O(1)$ – constant extra space.

3.2 Problem C2 – Count Elements in a Range Using Binary Search

Difficulty: Medium

Problem Statement

Given a sorted array and two values `low` and `high`, count how many elements fall in $[\text{low}, \text{high}]$ using binary search.

Pseudocode

```
// Binary search: find first index where arr[i] >= low_val
SET low = 0, high = n-1, firstIndex = n
WHILE low <= high DO
    mid = (low + high) / 2
    IF arr[mid] >= low_val THEN
        firstIndex = mid, high = mid - 1
    ELSE
        low = mid + 1

// Binary search: find last index where arr[i] <= high_val
SET low = 0, high = n-1, lastIndex = -1
WHILE low <= high DO
    mid = (low + high) / 2
    IF arr[mid] <= high_val THEN
        lastIndex = mid, low = mid + 1
    ELSE
        high = mid - 1

Count = lastIndex - firstIndex + 1
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {1, 3, 5, 7, 8, 9, 11, 14, 18};
    int n = 9;
    int low_val = 5;
    int high_val = 11;

    // Find first index >= low_val
    int low = 0, high = n - 1;
    int firstIndex = n;

    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] >= low_val) {
            firstIndex = mid;
            high = mid - 1;
        } else {
            low = mid + 1;
        }
```

```
    }

    // Find last index <= high_val
    low = 0;
    high = n - 1;
    int lastIndex = -1;

    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] <= high_val) {
            lastIndex = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    if (firstIndex > lastIndex) {
        cout << "No elements in range ["
             << low_val << ", "
             << high_val << "]" << endl;
        cout << "Count: 0" << endl;
    } else {
        int count = lastIndex - firstIndex + 1;
        cout << "Elements in range ["
             << low_val << ", "
             << high_val << "]: [";
        for (int i = firstIndex; i <= lastIndex; i++) {
            cout << arr[i];
            if (i < lastIndex)
                cout << ", ";
        }
        cout << "]" << endl;
        cout << "Count: " << count << endl;
    }

    return 0;
}
```

Output

```
Elements in range [5, 11]: [5, 7, 8, 9, 11]
Count: 5
```

Complexity Analysis

- **Time Complexity:** $O(\log n)$ – two binary searches.
- **Space Complexity:** $O(1)$ – constant extra space.

3.3 Problem C3 – Find Square Root Using Binary Search

Difficulty: Hard

Problem Statement

Without using `sqrt()` or `**`, use binary search to find the integer square root of n (largest x such that $x^2 \leq n$).

Pseudocode

```
IF n == 0 THEN answer = 0
SET low = 1, high = n, answer = 0
WHILE low <= high DO
    mid = (low + high) / 2
    IF mid <= n / mid THEN
        answer = mid
        low = mid + 1
    ELSE
        high = mid - 1
PRINT answer
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    long long n;
    cout << "Enter a number: ";
    cin >> n;

    if (n == 0) {
        cout << "Square root of 0 is 0" << endl;
        return 0;
    }

    long long low = 1;
    long long high = n;
    long long answer = 0;

    while (low <= high) {
        long long mid = (low + high) / 2;

        // mid <= n/mid avoids overflow
        if (mid <= n / mid) {
            answer = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    cout << "Square root of " << n
         << " is " << answer << endl;
```

```

    // Verify
    cout << answer << " * " << answer
         << " = " << answer * answer << endl;

    if (answer * answer == n) {
        cout << "Perfect square!" << endl;
    } else {
        cout << answer << "^2 = "
             << answer * answer << " <= "
             << n << " < "
             << (answer+1)*(answer+1)
             << " = " << (answer+1)
             << "^2" << endl;
    }

    return 0;
}
```

Output

```
Enter a number: 20
Square root of 20 is 4
4 * 4 = 16
4^2 = 16 <= 20 < 25 = 5^2
```

Complexity Analysis

- **Time Complexity:** $O(\log n)$ – binary search over $[0, n]$.
- **Space Complexity:** $O(1)$ – constant extra space.